

SocioLab — How the Simulation Actually Works

A mechanism reference for the local multi-agent social simulation engine.

SocioLab drops several LLM-driven characters — each with a private role, personality, goals, constraints, and secrets — into a shared social conflict, and streams a live, turn-by-turn transcript of how they behave under it. Every model runs locally through Ollama. This document walks through every layer: what the model is asked to do, what it's *not* trusted to do, and how the environment turns each turn into a real, measurable change in the simulated world.

1. The core design principle

Everything else in this document follows from one decision: **the model chooses an action; the server's environment engine — not the model — decides what that action does.** A model can say anything it wants, but it cannot invent how much tension rises, whether a proposal is "allowed," or when the simulation ends. Those are all deterministic rules in `server.js`.

This exists because small local models are unreliable narrators. Left unconstrained, a model will happily contradict itself — refuse a demand in its `message` while its `proposal` field simultaneously restates the very demand it just refused, or address the wrong participant, or repeat itself. SocioLab's architecture is built to make entire categories of that failure structurally impossible, rather than just asking nicely in the prompt.

2. End-to-end request flow

BROWSERUser fills in a scenario, 2–4 participants (each with a private character sheet), a round cap, and clicks *Run simulation*.



POST /API/SIMULATE `prepareSimulation()` validates the payload synchronously — scenario present, 2–4 valid agents with installed models, unique names. Any failure here returns a normal `400 JSON` error, no streaming involved yet.



STREAM OPENSOnce validated, the server sends `200` with `Content-Type: application/x-ndjson` and keeps the connection open. Every event from here on is one JSON object per line, flushed immediately — not buffered until the end.



RUNSIMULATION() LOOPFor round 1..N, for each agent in order: build that agent's prompt, call Ollama, validate/retry the response, compute the deterministic world-state effect, stream the result. (Full detail in §3–§5.)



BROWSERReads the stream with `response.body.getReader()`, splitting on newlines, dispatching each event to update the "thinking" indicator, append a transcript entry, update the world-state banner, and redraw the live analytics charts — all before the simulation itself has finished.

3. What each agent actually sees (and doesn't)

Every turn, exactly one agent gets a fresh prompt built by `agentPrompt()` . It's assembled from four distinct layers, and the separation between them is the whole point:

Layer	Visible to	Content
Private character sheet	Only this agent	Role, personality, primary goal, secondary goals, constraints, private information. Other agents' sheets are never included in this prompt.
Current world state	Everyone (identical)	Tension (0–10), authority attention (0–5), resolution status, each agent's public "position" label, each agent's standing (0–10), and any facts that have been REVEAL ed so far.
Conversation so far	Everyone (identical)	The last 5 public turns: who said what to whom, and which action they took.
Right now	Everyone (identical)	A direct quote of the single most recent turn, so the agent is anchored to respond to something specific rather than the conversation in the abstract.

The instructions layered on top of this (in the order they're actually sent — the target-naming rule deliberately sits *last*, immediately before the JSON is requested, because small models pay more attention to instructions near the end of a long prompt):

- Stay in character; talk like a real person, not a mediator or policy memo.
- React to the specific thing that was just said — don't restart or summarize the whole discussion.
- You are never obligated to agree, concede, or cooperate — judge the last message on its merits.
- Never reuse a sentence that already appears in the conversation.
- Speak only in first person; never write another character's line.
- Choose exactly one action from the fixed list (§4).
- (*last*) `target` must be exactly one other participant's name — never your own.

4. The fixed action space

The model doesn't write free-form intent — it must select one `action` from exactly ten options, and only five of them are allowed to carry a concrete `proposal`. This mapping is enforced twice: once in the prompt (so the model knows the rule), and again unconditionally in `parseAction()` server-side, which forces `proposal = null` for any ineligible action regardless of what text the model tried to put there.

Action	Meaning	Can carry a proposal?
SPEAK	A reaction or statement with no concrete ask	No
DEMAND	A one-sided order, no room for discussion	Yes
REFUSE	Flatly reject a demand or claim	No
NEGOTIATE	Open discussion, suggest a meeting or partial terms	Yes
COMPROMISE	Give up something of your own to reduce conflict	Yes
ACCEPT	Agree to what the other party wants	Yes
ESCALATE	Raise the stakes — threats, involving authority	Yes
ASK	Request information or clarification	No
REVEAL	Say a piece of your own private information out loud for the first time — it becomes permanently public	No (the fact itself is the message)
WITHDRAW	Disengage from the dispute entirely	No

Every turn's raw model output is a JSON object: `{ action, target, message, proposal }`. Ollama is called with a JSON-schema `format` constraint so the response is structurally valid JSON; SocioLab's own code then decides whether its *content* is trustworthy (§5).

5. Guardrails: what happens when the model misbehaves

Every candidate response goes through validation before it's accepted. If it fails, the server retries the same turn (up to 3 attempts, with rising temperature: 0.85 → 1.00 → 1.15) rather than silently keeping a bad response.

Check	What it catches	Where
Valid JSON, non-empty <code>message</code>	Malformed or empty output	<code>parseAction()</code>
Proposal eligibility	A <code>REFUSE</code> or <code>SPEAK</code> that tries to carry a contradictory proposal — forced to <code>null</code> unconditionally	<code>parseAction()</code>
Target validity	An invalid/missing target; a candidate with a genuinely valid target is always preferred over one without, across retries	<code>parseAction()</code> + retry loop
Banned generic phrases	Stock filler like "let's find a solution that works for everyone" — a fixed denylist	<code>isRejectableAction()</code>
Self-name mention	A character narrating about themselves in the third person (e.g. "Ben doesn't want to move")	<code>mentionsOwnName()</code>
Exact/near-duplicate detection	Copy-pasted or lightly reworded repeats of a recent line — both an exact-substring check and a fuzzy word-overlap check ($\geq 70\%$ shared words) against the last 6 turns	<code>wordOverlapRatio()</code>
Minimum length	Degenerate near-empty replies (e.g. "by tonight") that technically pass every other check	retry loop, requires ≥ 8 words

Last-resort fallback. If all 3 attempts fail every check, the server uses the best (longest, most-valid-target) candidate it saw across those attempts. Only if *nothing* usable was produced at all does it fall back to a single hardcoded line: "Give me a second, I need to think about how to say this." This has been rare in practice, but if that exact sentence ever appears in a transcript, that turn was not genuine model output.

6. The environment / effects engine

This is the part that is *never* generated by an LLM. `ACTION_EFFECTS` is a fixed lookup table: each of the 10 actions has a hardcoded effect on tension, authority risk, and the standing of both the acting agent and their target.

Action	Tension	Authority risk	Actor standing	Target standing
SPEAK	+0	+0	+0	+0
DEMAND	+2	+0	+1	-1
REFUSE	+1	+0	+1	+0
NEGOTIATE	-1	+0	+0	+0
COMPROMISE	-2	+0	-1	+1
ACCEPT	-2	+0	-2	+2
ESCALATE	+3	+2	+1	-1
ASK	+0	+0	+0	+0
REVEAL	+0	+0	+1	+0
WITHDRAW	-1	+0	-1	+0

Tension is clamped 0–10, authority risk 0–5, standing 0–10 per agent. **Standing is informational, not a win condition** — a participant's standing hitting 0 does not by itself end the simulation (an earlier version had this bug; it was removed). Every turn also snapshots the before/after values for tension, authority, and both agents' standing, so the transcript can show exactly what that specific turn changed.

How the simulation ends

Resolution	Trigger
Tentative agreement	Tension drops to ≤ 1 following a NEGOTIATE, COMPROMISE, or ACCEPT
Authority intervention likely	Authority risk reaches 5 (each ESCALATE adds 2)
"X has withdrawn from the dispute"	Any agent chooses WITHDRAW
Deadlock — no agreement reached after N rounds	The round cap is hit with none of the above having happened

Whichever condition fires first ends the round loop immediately — the simulation doesn't run the remaining scheduled rounds once resolved.

7. The streaming protocol

The HTTP response body is newline-delimited JSON (NDJSON), not a single blocking response. Five event types:

Event	Payload	Frontend reaction
status	{round, agent, message}	Shows a live "Asha is composing a response..." indicator with a pulsing dashed border, and updates the status badge
entry	The full turn: action, target, message, proposal, effect	Clears the thinking indicator, appends a transcript card, redraws the analytics charts
world	The complete world state after that turn	Updates the world-state banner (tension, authority, resolution, standing, revealed facts)
done	Final entries, agents, rounds completed, initial & final world	Finalizes the status line
error	An error message	Appended as a visible error card <i>without</i> erasing whatever transcript already streamed in

This exists so that a simulation that takes 30 seconds to several minutes (larger models are slow, and each turn can involve up to 3 retries) never looks frozen — every stage of "what's happening right now" is visible the moment it happens, not after the whole thing finishes.

8. The frontend

Setup panel

Scenario text, an experiment-design toggle (§9), a round cap (1–10), and 2–4 participant cards — each with name, model (pulled live from `GET /api/models`), role, personality, primary/secondary goals, constraints, and private information.

Discussion / Analytics tabs

The simulation output is split into two tabs so they don't compete for space: **Discussion** shows the live transcript (one card per turn: who, round, action → target badge, the message, the proposal if any, and the exact environment effect it caused); **Analytics** shows the three live charts (§9). Both update in the background regardless of which tab is active, so switching tabs mid-run always shows current data.

9. Live analytics

All three charts are computed client-side, purely by replaying the stream of `entry` events already received — no extra network calls. They redraw after every single turn.

1. **Social tension timeline** — a line chart of tension (0–10) across every turn, with round boundaries marked (Start, R1, R2...).
2. **Agent standing over time** — one line per participant, colored consistently with their card/transcript color, with sorted, non-overlapping end-of-line labels.
3. **Action distribution per participant** — a small bar-chart card per agent showing which actions they've actually used and how often — a behavioral fingerprint.

Design details worth noting: the four participant colors were run through an automated accessibility check (lightness band, colorblind-safe separation, contrast against the cream background) and corrected until all checks passed — the original teal/mustard/blue were too desaturated and one failed contrast. Every chart also has hover tooltips on each point/bar and a collapsed "View data" table underneath with the identical numbers, so nothing is locked inside the visualization only.

10. Experiment modes

Mode	What it tests
Role consistency	One model plays every participant. Tests whether a single model can sustain genuinely distinct personas simultaneously.
Model comparison	A different model per participant. Tests how model choice — not just the written persona — shapes behavior. Rotating which model plays which role across separate runs is how you tell "this model behaves aggressively" apart from "this role was written aggressively."

11. Known limitations

- **Small-model capability ceiling.** Sub-1B/1B models occasionally still mistarget or misclassify actions (e.g. calling a discussion-opener a DEMAND) even with all the guardrails above — those catch structural contradictions, not reasoning quality.
- **REVEAL is rarely used by choice.** The mechanic is fully functional — a revealed fact really does become permanently visible to every agent — but models tend to default to louder actions (DEMAND, ASK) rather than proactively choosing strategic disclosure, even when explicitly invited to consider it.
- **The scenario stays generic free text.** There is no per-scenario structured world model (e.g. an explicit budget, headcount, or inventory) — the environment's state variables (tension, authority, standing) are intentionally scenario-agnostic so the tool works on any conflict a user describes in plain language, at the cost of not modeling domain-specific stakes.

12. File map

server.js	HTTP server, Ollama adapter, prompt builder, guardrails, the environment/effects engine, NDJSON streaming.
public/index.html	Static markup: setup panel, tab structure, transcript/analytics containers, participant template.
public/app.js	All client-side behavior: form handling, the streaming reader, transcript/world-state rendering, the three chart builders.
public/styles.css	The visual theme — warm cream background, ink-black outlines, coral/teal/mustard/blue accent palette.

This document describes the system as implemented in the current codebase. It is a mechanism reference, not a claim of a validated behavioral benchmark — SocioLab produces reproducible, inspectable multi-agent transcripts under controlled incentives; it does not yet score model behavior against ground truth.